

Databricks full syllabus QnA

09/08/2026 00:44:49

<https://chatgpt.com/c/6a9f0a91-f828-83e8-a486-1f2193a78914>

Absolutely. Below is the complete interview-ready answer bank. I've ke...

Yes. These are the highest-priority 58, so here are concise, interview-...



Absolutely. Below is the **complete interview-ready answer bank**. I've kept the answers concise enough to revise quickly, but detailed enough to actually answer in an interview.

1. CLOUD PLATFORM

AWS Cloud Fundamentals

Basic

1. What is AWS, and why is it widely used for data engineering?

AWS is a cloud platform providing compute, storage, databases, networking, analytics, and AI services. It is widely used in data engineering because it is scalable, highly available, and provides managed services such as S3, Glue, Redshift, EMR, and Lambda.

2. What is the difference between an AWS Region and an Availability Zone?

A Region is a geographic area containing multiple Availability Zones. An Availability Zone is an isolated data center or group of data centers within a Region. Multiple AZs provide high availability and fault tolerance.

3. Explain Amazon EC2 and its common use cases.

EC2 provides virtual servers in the AWS cloud. It is commonly used for applications, custom data processing, ETL jobs, web servers, and workloads requiring control over the operating system and infrastructure.

4. What is the difference between EC2, Lambda, and ECS?

EC2 provides full control over virtual servers. Lambda runs code serverlessly without managing servers and is suited to short, event-driven tasks. ECS is a managed container orchestration service for running Docker containers.

5. What factors would you consider when choosing an EC2 instance type?

Consider CPU, memory, storage, network bandwidth, workload characteristics, expected traffic, scalability requirements, and cost. For example, compute-intensive workloads need compute-optimized instances.

6. What is Amazon S3, and how does its object storage model work?

S3 is AWS object storage designed for highly durable and scalable storage. Data is

stored as objects inside buckets, with each object consisting of data, metadata, and a unique key.

7. Explain S3 storage classes and when you would use them.

S3 provides classes such as Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, and Glacier Deep Archive. The choice depends mainly on access frequency, retrieval requirements, and cost.

8. What is the difference between S3 and EBS?

S3 is object storage accessed through APIs and is designed for scalable data storage. EBS provides block storage attached to EC2 instances and behaves more like a traditional disk.

9. What are the main types of databases available in AWS?

AWS provides relational databases such as RDS and Aurora, key-value databases such as DynamoDB, data warehouses such as Redshift, graph databases such as Neptune, and specialized databases for different workloads.

10. What is the difference between RDS, DynamoDB, and Redshift?

RDS is managed relational database hosting for transactional applications. DynamoDB is a highly scalable NoSQL key-value/document database. Redshift is a cloud data warehouse optimized for analytical queries.

Intermediate

11. How would you choose between EC2, Lambda, and ECS for a data processing workload?

Use Lambda for short, event-driven, serverless processing. Use ECS when the workload is containerized and requires more control or longer execution. Use EC2 when you need maximum control over infrastructure or specialized configurations.

12. How does auto scaling work in AWS?

Auto scaling automatically increases or decreases resources based on metrics or predefined policies. For example, EC2 Auto Scaling can add instances when CPU utilization increases and remove them when demand falls.

13. How would you design an S3 data lake with separate raw, processed, and curated data?

I would create separate logical zones or prefixes such as `raw/`, `processed/`, and

curated/. Raw data remains close to its source format, processed data is cleaned and transformed, and curated data is optimized for business consumption.

14. What is partitioning in S3, and why is it important for analytics?

Partitioning organizes data using attributes such as date, region, or department. It allows query engines to scan only relevant partitions, reducing processing time and cost.

15. Compare Amazon Redshift with a traditional relational database.

Redshift is designed for analytical workloads and large-scale aggregations using columnar storage and massively parallel processing. Traditional relational databases are generally optimized for transactional workloads.

16. What is VPC, and why is it important for data workloads?

A VPC is an isolated virtual network in AWS. It allows you to control IP ranges, subnets, routing, security groups, and network access for data processing and database workloads.

17. Explain public and private subnets, route tables, and Internet/NAT gateways.

A public subnet has a route to an Internet Gateway. A private subnet does not directly accept internet traffic. Route tables determine where traffic goes, while a NAT Gateway allows private resources to make outbound internet connections.

18. What is IAM, and how are users, groups, roles, and policies different?

IAM controls access to AWS resources. Users represent individual identities, groups organize users, roles provide temporary permissions to identities or services, and policies define what actions are allowed or denied.

19. What is CloudWatch, and how would you use it to monitor a data pipeline?

CloudWatch provides metrics, logs, dashboards, and alarms. I would use it to monitor job duration, failures, CPU/memory usage, error logs, and trigger alerts when thresholds are exceeded.

20. What are containers, and why would you use them for data engineering applications?

Containers package an application with its dependencies into a portable unit. They provide consistent environments, easier deployment, isolation, and reproducibility across development and production.

Advanced

21. An ETL job running on EC2 becomes slow as data volume increases. How would you investigate and improve it?

I would inspect CPU, memory, disk I/O, network usage, logs, and query performance. Then I would identify bottlenecks and consider parallel processing, better partitioning, optimized queries, more suitable instances, caching, or distributed processing.

22. How would you secure an S3-based data lake against unauthorized access?

Use IAM least privilege, bucket policies, Block Public Access, encryption, VPC endpoints where appropriate, logging, versioning, and monitoring. Sensitive data should have restricted access and encryption keys should be properly controlled.

23. How would you design an AWS data pipeline that is scalable, fault tolerant, and cost efficient?

I would use S3 for durable storage, managed or serverless processing where suitable, partitioned data formats such as Parquet, automated retries, monitoring, IAM controls, and autoscaling. I would also optimize storage classes and compute usage to control costs.

24. How would you troubleshoot an application that can access AWS services from one subnet but not another?

I would compare route tables, security groups, network ACLs, subnet configuration, NAT/Internet Gateway routes, DNS, and VPC endpoints between the subnets. I would also inspect flow logs and application errors.

25. How would you decide between serverless and container-based architecture for a production data pipeline?

I would consider execution duration, workload variability, dependencies, scaling requirements, operational complexity, and cost. Serverless is preferable for event-driven and variable workloads, while containers are better when applications require custom runtimes or long-running processes.

2. SQL

Introduction to SQL

Basic

26. What is the difference between WHERE and HAVING?

`WHERE` filters individual rows before grouping. `HAVING` filters groups after `GROUP BY` and is commonly used with aggregate functions.

27. Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

`INNER JOIN` returns matching rows from both tables. `LEFT JOIN` returns all rows from the left table plus matches. `RIGHT JOIN` does the opposite. `FULL OUTER JOIN` returns all rows from both tables, matching where possible.

28. What is the difference between UNION and UNION ALL?

`UNION` combines results and removes duplicates. `UNION ALL` combines results without removing duplicates and is generally faster.

29. What is a subquery, and when would you use one?

A subquery is a query nested inside another query. It is useful when the result of one query is required as input for another.

30. What are temporary tables, and when are they useful?

Temporary tables store intermediate results for a session or transaction. They are useful for breaking complex transformations into steps and reusing intermediate data.

31. What is NULL in SQL, and how does it behave in comparisons?

`NULL` represents an unknown or missing value. Comparisons with `NULL` generally return `UNKNOWN`, so `IS NULL` or `IS NOT NULL` should be used instead of `= NULL`.

32. How would you identify duplicate records in a table?

Use `GROUP BY` on the columns defining uniqueness and `HAVING COUNT(*) > 1`.

33. How would you remove duplicate records while keeping one record per entity?

Use `ROW_NUMBER()` partitioned by the entity's identifying columns, keep the row where `ROW_NUMBER() = 1`, and remove the remaining rows.

Intermediate

34. How would you clean inconsistent or invalid data using SQL?

Use functions such as `TRIM`, `LOWER`, `UPPER`, `CASE`, `COALESCE`, type conversions, validation conditions, and regular expressions where supported.

35. How would you handle NULL, blank, and invalid values differently?

`NULL` means the value is missing or unknown, while blank usually means an empty

string. Invalid values violate expected rules. I would identify and handle each separately based on business requirements.

36. Explain SQL window functions and why they are useful.

Window functions perform calculations across related rows without collapsing them into groups. They are useful for ranking, running totals, comparisons, and analytics.

37. What is the difference between ROW_NUMBER, RANK, and DENSE_RANK?

`ROW_NUMBER` gives every row a unique sequence. `RANK` gives tied rows the same rank and leaves gaps. `DENSE_RANK` gives tied rows the same rank without gaps.

38. How would you find the top N records within each group?

Use `ROW_NUMBER()`, `RANK()`, or `DENSE_RANK()` with `PARTITION BY group_column` and filter for the required rank.

39. How would you calculate running totals and moving averages?

Use window functions with `SUM()` or `AVG()` and an appropriate `ORDER BY` and window frame.

40. How would you compare the current row with the previous row using SQL?

Use the `LAG()` window function to access the previous row's value.

41. How would you find customers who made purchases in consecutive months?

Aggregate purchases by customer and month, then use `LAG()` to compare each purchase month with the previous purchase month and identify one-month differences.

Advanced

42. How would you optimize a query involving several large joins?

Filter data early, join on indexed or well-distributed keys, avoid unnecessary columns, inspect the execution plan, and ensure appropriate partitioning or clustering where supported.

43. What is an index, and how can indexes improve or hurt performance?

An index provides a faster lookup structure for data. It can significantly improve reads but increases storage and can slow inserts, updates, and deletes because indexes must also be maintained.

44. What is query execution planning, and how would you use an execution plan?

An execution plan shows how the database intends to execute a query. I would inspect scans, joins, estimated rows, indexes, sorting, and expensive operations to identify bottlenecks.

45. Why can a LEFT JOIN unexpectedly behave like an INNER JOIN?

If a condition on the right table is placed in the `WHERE` clause, rows with NULL matches are filtered out. Moving the condition into the `ON` clause can preserve LEFT JOIN behavior.

46. How would you optimize a query that scans billions of rows?

Filter early, use partition pruning, select only required columns, use appropriate indexes or clustering, optimize joins, and consider pre-aggregated tables or materialized views.

47. Explain common causes of SQL performance degradation and how you would troubleshoot them.

Common causes include missing indexes, large scans, poor joins, stale statistics, data skew, unnecessary sorting, and inefficient queries. I would use execution plans, query metrics, database monitoring, and workload analysis.

48. Write the approach for finding the second-highest salary without using LIMIT or TOP.

Use a ranking function such as `DENSE_RANK()` ordered by salary descending and select the row with rank 2. This also handles duplicate salaries correctly.

49. How would you identify gaps in a sequence of dates or IDs?

Use `LAG()` to compare each value with the previous value and identify differences greater than the expected increment.

50. How would you solve a complex business problem using CTEs and window functions?

I would break the problem into logical stages using CTEs, then use window functions for ranking, comparisons, running calculations, or deduplication. This makes the query easier to understand and maintain.

3. PYTHON FOR DATA ENGINEERS

Introduction to Python

Basic

51. What are the main built-in data structures in Python?

The main structures are lists, tuples, sets, and dictionaries. Each is suited to different requirements such as ordering, immutability, uniqueness, or key-value lookup.

52. Compare lists, tuples, sets, and dictionaries.

Lists are ordered and mutable. Tuples are ordered and immutable. Sets store unique values without relying on positional indexing. Dictionaries store key-value pairs.

53. What is the difference between mutable and immutable objects?

Mutable objects can be changed after creation, such as lists and dictionaries.

Immutable objects cannot be changed after creation, such as strings, integers, and tuples.

54. Explain if/elif/else and Python loops.

`if/elif/else` controls conditional execution. `for` loops iterate over collections or sequences, while `while` loops continue executing while a condition remains true.

55. What are list, dictionary, and set comprehensions?

Comprehensions provide a concise way to create collections from iterables, optionally applying conditions or transformations.

56. What is a function, and why should code be divided into functions?

A function is a reusable block of code that performs a specific task. Functions improve readability, testing, reuse, and maintainability.

**57. Explain positional arguments, keyword arguments, *args, and kwargs.

Positional arguments are passed by order, while keyword arguments are passed by parameter name. `*args` accepts multiple positional arguments and `**kwargs` accepts multiple keyword arguments.

58. What is exception handling, and how do try, except, else, and finally work?

`try` contains risky code, `except` handles errors, `else` runs when no exception occurs, and `finally` runs regardless of success or failure.

59. What is a Python module and package?

A module is a Python file containing reusable code. A package is a collection of related modules organized into a directory structure.

60. What is scripting in Python, and how is a Python script typically executed?

Python scripting means writing code to automate tasks. A script is commonly saved as a `.py` file and executed using the Python interpreter.

Intermediate

61. How does Python handle memory and object references?

Python manages memory automatically using mechanisms such as reference counting and garbage collection. Variables generally reference objects rather than directly storing the objects themselves.

62. What is the difference between shallow copy and deep copy?

A shallow copy copies the outer object but may share nested objects. A deep copy recursively copies nested objects as well.

63. How would you process a very large file without loading it entirely into memory?

Read the file incrementally using iterators, generators, or chunks. Process each portion and write results progressively rather than loading everything into memory.

64. How would you structure a Python ETL script for maintainability?

Separate extraction, transformation, validation, and loading into functions or modules. Add configuration management, logging, exception handling, testing, and clear documentation.

65. How would you log errors and execution information in a production Python pipeline?

Use Python's `logging` module with appropriate log levels such as INFO, WARNING, and ERROR. Include timestamps, job identifiers, useful context, and stack traces for failures.

66. What are NumPy arrays, and why are they generally more efficient than Python lists for numerical operations?

NumPy arrays store homogeneous data efficiently and support optimized vectorized operations implemented in compiled code, reducing Python-level loops.

67. What is vectorization in NumPy?

Vectorization means applying operations to entire arrays instead of explicitly looping

through individual elements. It usually improves performance and makes numerical code more concise.

68. What is a Pandas DataFrame, and how is it different from a Python dictionary?

A DataFrame is a two-dimensional tabular data structure with rows and columns. A dictionary primarily stores key-value mappings and does not inherently provide the same tabular operations.

69. How would you filter, transform, group, and aggregate data using Pandas?

Use boolean indexing for filtering, column operations or `apply` for transformations, `groupby()` for grouping, and functions such as `sum`, `mean`, and `count` for aggregation.

70. How would you handle missing and duplicate data in Pandas?

Use `isna()` to identify missing values and functions such as `fillna()` or `dropna()` to handle them. Use `duplicated()` and `drop_duplicates()` for duplicate records.

Advanced

71. How would you optimize a slow Pandas transformation?

Avoid Python loops and unnecessary `apply()` calls, use vectorized operations, reduce columns and data types, process in chunks, and consider alternatives such as Polars, Spark, or NumPy for larger workloads.

72. When would you use NumPy instead of Pandas?

Use NumPy for numerical computations and array-based operations where tabular labels are unnecessary. Use Pandas when working with structured tabular data and data manipulation.

73. How would you process a dataset larger than available memory using Python?

Process it in chunks, use generators, stream the data, or use distributed frameworks such as Spark. I would avoid loading the complete dataset into memory.

74. How would you design reusable Python components for multiple ETL pipelines?

Create modular functions and classes for common tasks such as extraction, validation, transformation, logging, and loading. Package them into reusable modules with configuration-driven behavior.

75. What Python practices would you follow when writing production-grade data engineering code?

Use clean modular code, type hints where useful, logging, exception handling, testing, configuration management, dependency management, documentation, code reviews, and version control.

4. EXPLORATORY DATA ANALYSIS AND DATA PROCESSING

Basic

76. What is Exploratory Data Analysis, and why is it performed?

EDA is the process of examining and summarizing data to understand its structure, patterns, relationships, anomalies, and quality before deeper analysis or modeling.

77. What is the difference between univariate, bivariate, and multivariate analysis?

Univariate analysis examines one variable. Bivariate analysis examines two variables and their relationship. Multivariate analysis examines multiple variables simultaneously.

78. Which graphical techniques would you use to understand a dataset?

Histograms and box plots help understand distributions. Scatter plots show relationships, bar charts compare categories, and line charts show trends over time.

79. What are outliers, and how can you identify them?

Outliers are observations that differ substantially from typical values. They can be identified using box plots, IQR, z-scores, statistical methods, or domain-specific rules.

80. What is correlation, and why does correlation not necessarily imply causation?

Correlation measures the strength and direction of association between variables. Two variables can be correlated because of coincidence, reverse causality, or a third factor, without one causing the other.

Intermediate

81. What is dimensionality reduction, and why is it useful?

Dimensionality reduction reduces the number of features while preserving important

information. It can improve visualization, reduce computational cost, and sometimes reduce noise.

82. Explain PCA at a high level.

Principal Component Analysis transforms correlated variables into a smaller number of uncorrelated principal components that capture the maximum possible variance.

83. What is hypothesis testing?

Hypothesis testing is a statistical method for determining whether observed evidence is strong enough to reject a proposed assumption about a population.

84. Explain null and alternative hypotheses.

The null hypothesis represents the default assumption, usually that there is no significant effect or difference. The alternative hypothesis represents the effect or difference being investigated.

85. What are p-values and significance levels?

A p-value measures how compatible the observed data is with the null hypothesis. A significance level, such as 0.05, is a predefined threshold used to decide whether evidence is sufficient to reject the null hypothesis.

86. How would you choose an appropriate statistical test for a problem?

Consider the research question, variable types, number of groups, independence, distribution assumptions, and sample size. The test should match both the data and the question.

87. How would you aggregate data in Pandas?

Use `groupby()` with aggregation functions such as `sum()`, `mean()`, `count()`, `min()`, and `max()`.

88. How would you create visualizations to communicate analytical findings?

Choose a chart based on the question, simplify unnecessary elements, label axes clearly, highlight important findings, and provide context so the visualization is easy to interpret.

89. Why is normalization or standardization sometimes required before machine learning?

Features can have very different scales. Scaling prevents large-valued features from dominating algorithms and can improve optimization for models sensitive to feature magnitude.

90. Compare min-max scaling, standardization, and robust scaling.

Min-max scaling typically maps values to a fixed range such as 0 to 1. Standardization centers data around zero with unit variance. Robust scaling uses statistics such as median and IQR and is less sensitive to outliers.

5. PYTHON WITH SQL DATABASES

Basic

91. How can Python connect to a SQL database?

Python can connect using database-specific drivers or libraries such as `psycopg2`, `pyodbc`, SQLAlchemy, or database-specific connectors.

92. What is a database connection and cursor?

A connection represents communication with the database. A cursor is used to execute SQL statements and retrieve results through that connection.

93. How would you execute SQL queries from Python?

Create a database connection, create or obtain a cursor, execute the SQL using the cursor, retrieve results if required, and commit or roll back the transaction.

94. How would you insert a Pandas DataFrame into a SQL database?

Pandas provides `DataFrame.to_sql()`, commonly used with SQLAlchemy connections. For large datasets, bulk-loading or database-specific loading methods are usually more efficient.

95. How would you safely handle database errors and transactions in Python?

Use exception handling, transactions, commits, and rollbacks. Connections should be closed properly, preferably using context managers.

Intermediate

96. How would you build a Python program that extracts data from SQL, transforms it, and writes it back?

Create separate extraction, transformation, validation, and loading functions. Use parameterized queries, transactions, logging, configuration, and error handling.

97. What is parameterized SQL, and why should you use it from Python?

Parameterized SQL separates SQL structure from user-provided values. It helps

prevent SQL injection and handles values safely.

98. How would you process large database results efficiently in Python?

Read results in batches or chunks rather than loading everything into memory. Push filtering and aggregation into SQL whenever practical.

99. How would you manage database connections in a production application?

Use connection pooling, secure credential management, timeouts, retries where appropriate, proper cleanup, and monitoring.

100. How would you troubleshoot a Python ETL job that is timing out while reading from a database?

Inspect the SQL execution plan, indexes, filters, network latency, database load, connection settings, and result size. I would also check whether processing can be pushed into SQL or performed in batches.

6. BIG DATA, DATA LAKES AND NoSQL

Basic

101. What is Big Data?

Big Data refers to datasets whose size, speed, complexity, or variety makes them difficult to process efficiently using traditional systems.

102. Explain the main characteristics of Big Data.

Common characteristics are volume, velocity, variety, veracity, and value.

103. Why do traditional systems struggle with very large datasets?

They often have limited vertical scalability, storage capacity, processing power, and fault tolerance. Distributed systems can spread storage and computation across many machines.

104. What is a data lake?

A data lake is a centralized storage system that can store large amounts of structured, semi-structured, and unstructured data, often in its original form.

105. What is data ingestion?

Data ingestion is the process of collecting and moving data from source systems into a storage or processing platform.

106. Compare batch and streaming ingestion.

Batch ingestion processes data periodically in groups. Streaming ingestion processes data continuously or in near real time as events arrive.

Intermediate

107. What are the major components of a Big Data processing system?

Typical components include data sources, ingestion systems, distributed storage, processing engines, metadata/catalog services, orchestration, and serving or analytics layers.

108. Compare distributed storage and distributed processing.

Distributed storage spreads data across multiple machines for scalability and fault tolerance. Distributed processing divides computation across multiple machines to process large datasets efficiently.

109. Explain the roles of ingestion, storage, processing, and serving layers.

Ingestion collects data, storage persists it, processing transforms or analyzes it, and serving makes processed data available to applications, analytics, or users.

110. When would you use a NoSQL database instead of a relational database?

Use NoSQL when you need flexible schemas, massive horizontal scaling, very high throughput, or data models that do not fit relational structures well.

Advanced

111. How would you design a scalable ingestion system for high-volume data?

Use distributed ingestion services, partition data by suitable keys, buffer incoming events, support parallel consumers, make processing idempotent, and monitor throughput and failures.

112. How would you choose between batch and streaming architecture?

Choose streaming when low latency is a business requirement. Choose batch when data can be processed periodically and simplicity and cost efficiency are more important.

113. What challenges arise when processing Big Data across distributed systems?

Challenges include network overhead, data skew, node failures, synchronization, serialization, partitioning, shuffling, consistency, and resource management.

114. Compare document, key-value, column-family, and graph NoSQL databases.

Document databases store flexible documents. Key-value databases map keys to values. Column-family databases organize data into distributed column groups. Graph databases represent entities and relationships as nodes and edges.

115. What factors determine whether a workload should use SQL, NoSQL, or a data lake?

Consider data structure, query patterns, transaction requirements, scale, latency, schema flexibility, consistency, cost, and analytical requirements.

7. AI, ML, DL AND GENAI

AI, ML and DL Fundamentals

Basic

116. What is Artificial Intelligence?

AI is the broader field of creating systems that can perform tasks that normally require human intelligence, such as reasoning, perception, learning, and decision-making.

117. What is Machine Learning?

Machine Learning is a subset of AI where systems learn patterns from data and use those patterns to make predictions or decisions.

118. What is the difference between supervised, unsupervised, and reinforcement learning?

Supervised learning uses labeled data. Unsupervised learning finds patterns in unlabeled data. Reinforcement learning learns actions through interaction with an environment and rewards.

119. What is Deep Learning?

Deep Learning is a subset of machine learning that uses multi-layer neural networks to learn complex patterns from large amounts of data.

120. How are AI, ML, Deep Learning, and Generative AI related?

AI is the broadest field. ML is a subset of AI, Deep Learning is a subset of ML, and

Generative AI refers to models capable of generating new content such as text, images, audio, or code.

121. What is a training dataset, validation dataset, and test dataset?

Training data is used to learn model parameters. Validation data is used to tune models and hyperparameters. Test data provides a final unbiased evaluation.

122. What is overfitting and underfitting?

Overfitting occurs when a model learns training data too closely and performs poorly on unseen data. Underfitting occurs when the model is too simple to capture important patterns.

Intermediate

123. What is feature engineering?

Feature engineering is the process of creating, transforming, or selecting input variables to help a machine learning model learn useful patterns.

124. Explain classification and regression.

Classification predicts categories, such as fraud or not fraud. Regression predicts continuous numerical values, such as revenue or temperature.

125. What is clustering, and when would you use it?

Clustering groups similar data points without predefined labels. It can be used for customer segmentation, anomaly detection, or exploratory analysis.

126. What is model evaluation, and why should the evaluation metric match the business problem?

Model evaluation measures how well a model performs. The metric should reflect business consequences because accuracy alone may be misleading for problems such as fraud detection or medical diagnosis.

127. Compare precision, recall, F1 score, and accuracy.

Accuracy measures overall correct predictions. Precision measures how many predicted positives are actually positive. Recall measures how many actual positives were found. F1 balances precision and recall.

128. What is gradient descent?

Gradient descent is an optimization algorithm that iteratively adjusts model parameters to minimize a loss function.

129. What is a neural network?

A neural network is a model composed of interconnected computational units organized into layers that learn relationships between inputs and outputs.

130. Explain neurons, weights, biases, and activation functions.

A neuron combines inputs using weights, adds a bias, and applies an activation function. Weights determine input importance, while bias shifts the activation.

131. Why are activation functions required in neural networks?

They introduce non-linearity, allowing neural networks to learn complex relationships that cannot be represented effectively using only linear operations.

132. What is backpropagation?

Backpropagation calculates how much each model parameter contributed to the error and uses gradients to update those parameters during training.

RNNs, LSTMs and Transformers

Basic

133. What problem are RNNs designed to solve?

RNNs are designed to process sequential data while maintaining information from previous steps, making them useful for tasks involving time or sequence.

134. What are the limitations of traditional RNNs?

They can suffer from vanishing and exploding gradients, making it difficult to learn long-term dependencies. They are also difficult to parallelize across sequence positions.

135. What is an LSTM, and how does it improve upon an RNN?

LSTM is a type of RNN that uses gates and a cell state to control information flow. This helps it retain important information over longer sequences.

136. What is a Transformer?

A Transformer is a neural network architecture based primarily on attention mechanisms rather than recurrent processing. It is widely used for language and other sequence tasks.

137. Why did Transformers become important for NLP?

They handle long-range dependencies effectively and allow substantial parallelization during training, enabling large-scale models to learn complex language patterns.

Intermediate

138. Explain attention at a high level.

Attention allows a model to determine which parts of the input are most relevant when processing a particular element.

139. What is self-attention?

Self-attention allows each token in a sequence to consider other tokens in the same sequence and calculate their relevance.

140. What are tokens and embeddings?

Tokens are units into which input text is divided. Embeddings are numerical vector representations of tokens that capture semantic or contextual information.

141. How can Transformers be used for text translation?

A Transformer can encode the source language and generate the target language while using attention to understand relationships between words across the sequence.

142. How can Transformers be used for sentiment analysis?

A pretrained Transformer can be fine-tuned or prompted to classify text into sentiment categories such as positive, negative, or neutral.

Advanced

143. Compare RNNs, LSTMs, and Transformers for sequence processing.

RNNs process sequences sequentially and struggle with long dependencies. LSTMs improve memory through gating but remain sequential. Transformers use attention and can process sequence positions more parallelly.

144. Why can Transformers process sequences more efficiently than traditional RNNs?

Transformers can process many sequence positions in parallel during training, whereas RNNs typically process tokens sequentially.

145. What are the computational challenges of self-attention?

Standard self-attention has computational and memory costs that grow significantly with sequence length, making very long contexts expensive.

146. How would you choose an NLP model for a production use case?

Consider task requirements, accuracy, latency, cost, model size, context length, privacy, deployment environment, and available infrastructure.

8. CONVOLUTIONAL NEURAL NETWORKS

Basic

147. How is an image represented as data?

An image is represented as a grid of pixels. Each pixel contains numerical values representing intensity or color channels such as RGB.

148. What is a Convolutional Neural Network?

A CNN is a neural network architecture designed to learn spatial patterns in data, especially images, using convolutional layers.

149. What is a convolution/filter in a CNN?

A filter is a small matrix of learned weights that moves across an image and detects patterns such as edges, textures, or shapes.

150. What are pooling layers?

Pooling reduces the spatial dimensions of feature maps while retaining important information. Common approaches include max pooling and average pooling.

151. Why are CNNs effective for image processing?

They exploit spatial structure, use shared weights, and progressively learn features from simple patterns to complex objects.

Intermediate

152. How does a CNN progressively learn image features?

Early layers often learn simple features such as edges. Deeper layers combine these into textures, shapes, parts, and eventually higher-level objects.

153. What is the difference between image classification and object detection?

Classification predicts what an image contains. Object detection identifies what objects exist and where they are, usually using bounding boxes.

154. How can CNNs be used for facial detection?

A CNN can learn visual features associated with faces and classify image regions as containing a face or not. More advanced systems can also locate multiple faces.

155. What challenges occur when using CNNs for real-world image recognition?

Challenges include varying lighting, scale, rotation, occlusion, background complexity, image quality, dataset bias, and changes between training and production environments.

Advanced

156. How could CNN-based computer vision be used in self-driving cars?

CNNs can process camera images to detect lanes, vehicles, pedestrians, traffic signs, and other objects. Their outputs can contribute to the vehicle's perception system.

157. What factors affect the performance of an image recognition model?

Dataset quality and size, model architecture, image resolution, augmentation, training strategy, class balance, hardware, and how closely production data matches training data.

158. How would you evaluate a computer vision model in production?

Use metrics appropriate to the task, such as accuracy, precision, recall, F1, IoU, or mAP. Also monitor latency, false positives, false negatives, data drift, and real-world business outcomes.

9. REINFORCEMENT LEARNING

Basic

159. What is Reinforcement Learning?

Reinforcement Learning is a machine learning approach where an agent learns how to act by interacting with an environment and receiving rewards or penalties.

160. What are an agent, environment, state, action, and reward?

The agent makes decisions. The environment is the system it interacts with. A state represents the current situation, an action is a decision, and a reward indicates how desirable the outcome was.

161. What is a Q-function?

A Q-function estimates the expected future reward of taking a particular action in a particular state and then following a policy.

162. What is meant by optimal action?

An optimal action is the action expected to maximize cumulative future reward from the current state.

Intermediate

163. Explain Q-learning.

Q-learning is a value-based reinforcement learning algorithm that learns the expected value of state-action pairs and uses those values to select actions.

164. What is the difference between exploration and exploitation?

Exploration means trying actions to discover potentially better strategies. Exploitation means choosing actions that are already believed to produce good rewards.

165. What is Deep Q-Networks?

A DQN uses a neural network to approximate the Q-function, allowing Q-learning to handle large or complex state spaces.

166. What are the limitations of Q-learning?

It can struggle with very large or continuous action spaces, can be unstable with function approximation, and may require substantial training data.

Advanced

167. What are policy gradient methods?

Policy gradient methods directly optimize a parameterized policy by adjusting its parameters toward actions that produce higher expected rewards.

168. Compare value-based and policy-based reinforcement learning.

Value-based methods learn the value of actions and derive a policy from them. Policy-based methods directly learn the policy. Policy methods can handle continuous action spaces more naturally.

169. When would DQN be unsuitable for a problem?

DQN is generally unsuitable when actions are continuous or extremely large in number, or when the environment requires a different approach to stable policy optimization.

170. How would you evaluate a reinforcement learning system?

Evaluate average cumulative reward, success rate, stability, sample efficiency, safety, generalization, and performance across different environments or scenarios.

10. AGENTIC AI

Basic

171. What is Agentic AI?

Agentic AI refers to AI systems that can autonomously pursue goals by planning tasks, using tools, making decisions, observing results, and taking further actions.

172. How is an AI agent different from a traditional chatbot?

A chatbot primarily responds to user messages. An agent can reason through multi-step tasks, use external tools, access information, and take actions toward a goal.

173. What are the key components of an agentic AI system?

Typical components include an AI model, instructions or goals, memory or state, planning/reasoning, tools, knowledge sources, an execution loop, and safety controls.

174. What is tool use in an AI agent?

Tool use allows an agent to call external functions such as APIs, databases, search systems, calculators, or enterprise applications to obtain information or perform actions.

175. What is the role of planning and reasoning in an agent?

Planning breaks a complex goal into steps, while reasoning helps select appropriate actions and adapt based on results.

Intermediate

176. What types of business problems are suitable for Agentic AI?

Problems involving multi-step workflows, information retrieval, decision support, automation, customer service, operations, research, and repetitive tasks that require interaction with multiple systems.

177. How can Agentic AI interact with databases, APIs, and enterprise systems?

Agents can use controlled tools or APIs that expose specific operations. Authentication, authorization, validation, and access controls should be enforced around these tools.

178. What are the risks of giving an AI agent access to tools?

Risks include unauthorized actions, data leakage, incorrect decisions, excessive permissions, prompt injection, and unintended side effects. Tool access should follow least privilege and include safeguards.

179. How would you evaluate an agentic AI solution?

Evaluate task success, accuracy, reliability, tool-use correctness, latency, cost, safety,

failure recovery, and user satisfaction using representative test scenarios.

180. How would you explain the value of an Agentic AI solution to a client?

I would connect the technology to measurable outcomes such as reduced manual effort, faster processing, fewer errors, improved customer experience, or lower operating costs.

11. PRODUCTIVITY TOOLS

GitHub Copilot

Basic

181. What is GitHub Copilot?

GitHub Copilot is an AI-powered coding assistant that helps developers write, explain, modify, and understand code.

182. How does Copilot assist with code completion?

It analyzes surrounding code and context and suggests code completions, functions, or larger code sections based on the developer's intent.

183. How can prompts be used to generate code with Copilot?

Developers can describe the desired behavior in natural language and provide relevant context, constraints, inputs, and expected outputs to improve generated code.

184. What are the limitations of AI-generated code?

Generated code can contain bugs, security vulnerabilities, inefficient logic, outdated approaches, or incorrect assumptions. Developers must review and test it.

Intermediate

185. How would you use Copilot to understand an unfamiliar codebase?

I would ask it to explain files, functions, dependencies, control flow, and architecture, then verify its explanations against the actual code.

186. How would you verify code generated by Copilot?

Review the logic, run tests, inspect edge cases, check security and performance, and compare the implementation against requirements and documentation.

187. How can Copilot improve developer productivity without replacing engineering judgment?

It reduces repetitive coding and accelerates exploration, documentation, debugging, and implementation. Engineers still make architectural, security, quality, and business decisions.

Microsoft 365 Copilot

Basic

188. What is Microsoft 365 Copilot?

Microsoft 365 Copilot is an AI assistant integrated into Microsoft 365 applications such as Word, Excel, PowerPoint, Outlook, and Teams.

189. What are the core capabilities of Microsoft 365 Copilot?

It can summarize information, generate and edit content, analyze data, draft communications, extract insights, and assist with productivity tasks.

190. How can Copilot assist with Word, Excel, and email?

It can draft and summarize documents in Word, analyze and explain data in Excel, and draft or summarize emails in Outlook.

191. What makes a good business prompt for Copilot?

A good prompt clearly states the objective, context, audience, constraints, desired format, and relevant data or source material.

192. What risks should organizations consider when using AI productivity tools?

Organizations should consider data privacy, confidentiality, incorrect outputs, access control, compliance, intellectual property, and overreliance on AI-generated content.

Intermediate

193. How could Copilot automate repetitive email-related tasks?

It can summarize long email threads, draft responses, extract action items, and help prioritize or organize communications.

194. How could Copilot help analyze business data in Excel?

It can identify trends, summarize data, create formulas, suggest visualizations, and answer questions about the dataset using natural language.

195. How would you write an effective prompt for generating a professional business document?

Specify the document purpose, audience, tone, key points, structure, length, source information, and any formatting requirements.

12. DATA WAREHOUSING FUNDAMENTALS

Basic

196. What is a Data Warehouse?

A data warehouse is a centralized system designed to store integrated historical data for reporting, analytics, and business intelligence.

197. What are the main characteristics of a Data Warehouse?

It is generally subject-oriented, integrated, time-variant, and designed to support analytical rather than transactional workloads.

198. Compare OLTP and OLAP systems.

OLTP systems handle frequent transactions and are optimized for fast inserts and updates. OLAP systems handle complex analytical queries over large amounts of historical data.

199. What are fact and dimension tables?

Fact tables contain measurable business events such as sales or transactions.

Dimension tables contain descriptive information such as customer, product, or date.

200. What is a star schema?

A star schema has a central fact table connected directly to denormalized dimension tables. It is simple and efficient for analytical queries.

201. What is a snowflake schema?

A snowflake schema normalizes dimension tables into additional related tables. It reduces redundancy but increases query complexity.

202. What are common objects found in a data warehouse?

Common objects include tables, views, materialized views, schemas, dimensions, facts, indexes or clustering structures, stored procedures, and metadata objects.

Intermediate

203. Compare ETL and ELT.

ETL transforms data before loading it into the target system. ELT loads raw data first and performs transformations inside the target platform.

204. Explain the major layers of a typical Data Warehouse architecture.

A typical architecture contains source systems, ingestion/staging, transformation, warehouse storage, semantic or serving layers, and reporting/analytics tools.

205. What is data modeling?

Data modeling is the process of designing how data entities, attributes, relationships, and business rules are represented and stored.

206. Compare conceptual, logical, and physical data models.

A conceptual model provides a high-level business view. A logical model defines entities, attributes, and relationships. A physical model defines implementation details such as tables, columns, indexes, and data types.

207. What are one-to-one, one-to-many, and many-to-many relationships?

One-to-one means one record relates to one record. One-to-many means one record can relate to many. Many-to-many means multiple records can relate to multiple records, usually through a bridge table.

208. What are primary keys and foreign keys?

A primary key uniquely identifies a record. A foreign key references a key in another table and represents a relationship between tables.

209. Explain the typical data model development lifecycle.

It generally involves gathering requirements, identifying entities and relationships, creating conceptual and logical models, designing the physical model, implementing it, testing it, and refining it based on usage.

210. What is dimensional data modeling?

Dimensional modeling organizes data into fact and dimension tables to make analytical queries simple, understandable, and performant.

211. What is a surrogate key, and why is it useful?

A surrogate key is an artificial identifier, usually numeric, assigned to a warehouse record. It provides stable warehouse-specific identity and helps manage historical dimension changes.

212. What are slowly changing dimensions?

SCDs are techniques for handling changes to dimension attributes over time. Common types include Type 1, which overwrites values, and Type 2, which preserves historical versions.

Advanced

213. How would you choose between a star and snowflake schema?

I would choose a star schema for simpler queries and faster analytics when denormalization is acceptable. Snowflake can be useful when dimensions are large, hierarchical, or require reduced redundancy.

214. How would you model a sales analytics system?

Create a sales fact table containing measures such as quantity and revenue, connected to dimensions such as customer, product, store, employee, and date.

215. How would you handle historical changes in customer information?

Use an appropriate SCD strategy. For important historical reporting, Type 2 can preserve previous versions with effective dates and current-record indicators.

216. What factors should be considered when designing a warehouse for large analytical workloads?

Consider data volume, query patterns, concurrency, partitioning, distribution, storage format, indexing or clustering, workload isolation, scalability, security, and cost.

217. How does NoSQL data modeling differ from relational data modeling?

Relational modeling generally emphasizes normalized entities and relationships. NoSQL modeling is often driven by access patterns and may intentionally duplicate data for fast reads.

218. When would denormalization be beneficial in an analytical system?

Denormalization can reduce joins and simplify frequent analytical queries, improving read performance when additional storage and data duplication are acceptable.

219. How would poor data modeling affect query performance and data quality?

Poor modeling can cause excessive joins, duplication, inconsistent definitions, difficult maintenance, slow queries, and incorrect analytical results.

220. How would you design a model that supports both historical reporting and near-real-time analytics?

Use a layered architecture where streaming or micro-batch data supports low-latency access while historical data is stored in an analytical warehouse or lakehouse. Shared business definitions should keep both views consistent.

13. DATA INTEGRATION AND TALEND

Basic

221. What is Data Integration?

Data integration combines data from multiple sources and makes it available in a consistent format for processing, analytics, or applications.

222. Why is data integration important in data engineering?

Organizations typically have data spread across databases, applications, files, APIs, and other systems. Integration brings this data together for reliable analysis and decision-making.

223. What is Talend?

Talend is a data integration and data management platform used to build pipelines for extracting, transforming, loading, and managing data.

224. What is a Talend job?

A Talend job is a visual workflow consisting of connected components that perform tasks such as reading data, transforming it, validating it, and writing results.

225. What are Talend components?

Components are reusable building blocks that perform specific operations, such as reading databases, filtering records, transforming fields, or writing files.

Intermediate

226. How would you build an ETL pipeline using Talend?

Connect source components to transformation components and then to target components. Add validation, error handling, parameters, logging, and scheduling as required.

227. How do Talend components connect and pass data between each other?

Components are connected using links that define data flow or control flow. Row connections can pass records between components, while triggers can control execution order.

228. How would you handle errors and invalid records in a Talend job?

Use reject flows, validation components, exception handling, logging, and separate error outputs. Invalid records can be stored for investigation rather than silently discarded.

229. How would you parameterize a Talend job for different environments?

Externalize environment-specific values such as database URLs, credentials, file paths, and configuration settings using context variables.

230. How would you deploy and schedule a Talend job?

Build or publish the job, deploy it to the target environment, configure required parameters and dependencies, and schedule it through the appropriate Talend or enterprise scheduling mechanism.

14. PYSPARK

Spark Fundamentals

Basic

231. What is Apache Spark, and why is it used for Big Data processing?

Apache Spark is a distributed processing engine designed to process large datasets across clusters. It provides scalable computation and supports SQL, batch processing, streaming, machine learning, and graph workloads.

232. Explain Spark's basic cluster architecture.

A Spark application consists of a driver and executors running across a cluster. The driver coordinates execution while executors perform distributed tasks.

233. What are the Driver, Executors, and Cluster Manager?

The Driver coordinates the application and creates the execution plan. Executors run tasks and store data. The Cluster Manager allocates resources to Spark applications.

234. What are Spark's major services and components?

Major components include Spark Core, Spark SQL, Structured Streaming, MLlib, and GraphX. Spark applications can also integrate with various storage and cluster systems.

235. What is the Spark ecosystem?

The Spark ecosystem includes Spark Core, Spark SQL, Structured Streaming, MLlib, GraphX, APIs such as PySpark, and integrations with distributed storage and cluster platforms.

236. What is distributed data processing?

It means splitting data and computation across multiple machines so large workloads can be processed in parallel.

237. How is PySpark different from standard Python data processing?

Standard Python generally processes data on a single machine unless additional distributed tools are used. PySpark allows Python code to orchestrate distributed processing using Apache Spark.

238. What is an RDD?

An RDD is a Resilient Distributed Dataset, a fault-tolerant distributed collection of data that can be processed in parallel across a cluster.

239. What are transformations and actions in Spark?

Transformations create new datasets from existing ones, such as `filter` and `map`. Actions trigger execution and return results or write data, such as `count` and `save`.

240. What is lazy evaluation?

Spark does not immediately execute transformations. It builds a logical execution plan and executes it when an action is called, allowing optimization.

Intermediate

241. What is RDD lineage, and why is it important?

Lineage records the sequence of transformations used to create an RDD. Spark can use lineage to recompute lost partitions instead of requiring complete data replication.

242. When would you use an RDD instead of a DataFrame?

RDDs may be appropriate for low-level transformations or data that does not fit naturally into a structured schema. DataFrames are generally preferred for structured data because Spark can optimize them.

243. What is a Spark DataFrame?

A Spark DataFrame is a distributed collection of structured data organized into named columns, similar conceptually to a database table.

244. Compare RDDs, DataFrames, and Spark SQL.

RDDs provide low-level distributed data processing. DataFrames provide structured APIs with optimization. Spark SQL allows SQL-based querying over structured Spark data.

245. What are narrow and wide transformations?

Narrow transformations do not require data to move between partitions. Wide transformations require data redistribution, usually causing a shuffle.

246. What causes a shuffle in Spark?

Operations such as `groupBy`, `join`, `distinct`, and `orderBy` can require data to move between partitions, resulting in a shuffle.

247. What is Spark partitioning?

Partitioning divides distributed data into chunks that can be processed independently by executors.

248. How would you optimize a Spark job suffering from excessive shuffling?

Reduce unnecessary wide transformations, filter early, choose appropriate partitioning, use broadcast joins for small tables, and avoid unnecessary repartitioning.

249. What are broadcast joins, and when should they be used?

A broadcast join sends a small dataset to each executor so it can join locally with a large dataset. It is useful when the smaller dataset fits comfortably in executor memory.

250. What is caching/persistence in Spark, and when is it useful?

Caching stores computed data in memory or another storage level so it can be reused without recomputation. It is useful when the same dataset is used multiple times.

251. How does Spark handle failures in distributed processing?

Spark can recompute lost partitions using lineage. It can also retry failed tasks and leverage cluster-level fault tolerance mechanisms.

252. How would you identify a data skew problem in Spark?

Look for partitions or tasks that take significantly longer than others. Spark UI metrics, partition-size analysis, and key-frequency analysis can reveal skew.

Advanced

253. How would you optimize a slow PySpark job?

Inspect the Spark UI, identify slow stages, reduce shuffles, optimize joins, filter early, select required columns, use appropriate partitions, avoid unnecessary UDFs, and use caching only where beneficial.

254. What factors determine the number of partitions?

Consider input size, cluster resources, number of cores, data distribution, operation type, and target partition size.

255. How can poor partitioning affect Spark performance?

Too few partitions underutilize the cluster. Too many create scheduling and metadata overhead. Uneven partitions cause data skew and straggler tasks.

256. How would you troubleshoot an executor running out of memory?

Inspect executor logs and Spark UI, check partition sizes and skew, reduce data held in memory, avoid collecting large datasets, optimize joins, increase resources if necessary, and adjust partitioning.

257. What is Spark Structured Streaming?

Structured Streaming is Spark's stream-processing engine built on the DataFrame and SQL APIs. It processes continuously arriving data using declarative transformations.

258. Compare batch processing with Spark Structured Streaming.

Batch processing handles a bounded dataset. Structured Streaming handles continuously arriving data while using many of the same DataFrame and SQL APIs.

259. What are checkpoints in Spark Streaming?

Checkpoints persist information required to recover streaming query state and progress after failures.

260. How would you design an incremental ETL pipeline using PySpark?

Identify new or changed records using timestamps, change tracking, or another reliable incremental key. Process only those records and write them idempotently to the target.

261. How would you validate data quality during a PySpark ETL pipeline?

Check schema, null rates, uniqueness, valid ranges, referential integrity, record counts, duplicates, and business rules. Invalid records should be isolated or handled according to requirements.

262. How would you process synthetic data using PySpark?

Generate or load synthetic data, define the schema, distribute it across Spark partitions, apply transformations, validate it, and write the output to the required storage format.

263. What is the difference between synthetic, anonymized, and real data?

Real data comes from actual systems. Anonymized data is real data modified to reduce the ability to identify individuals. Synthetic data is artificially generated rather than directly representing real records.

264. What techniques can be used to generate synthetic tabular data?

Use random generation, statistical distributions, rule-based generation, sampling, or machine-learning-based synthetic data generators.

265. How would you generate synthetic semi-structured data?

Define the required schema and relationships, generate records programmatically, and serialize them into formats such as JSON with realistic nested structures and distributions.

266. What validation checks would you perform on a synthetic dataset?

Check schema, data types, ranges, uniqueness, distributions, relationships, null rates, duplicates, referential integrity, and whether the dataset satisfies business constraints.

267. How would you scale a PySpark ETL pipeline as the dataset grows?

Increase parallelism appropriately, optimize partitioning, use efficient formats such as Parquet, reduce shuffles, optimize joins, and scale cluster resources when necessary.

268. What best practices would you follow when building production PySpark pipelines?

Use modular code, schema validation, efficient transformations, partitioning, monitoring, logging, data-quality checks, idempotency, proper error handling, configuration management, and automated testing.

15. DATABRICKS

Databricks Fundamentals

Basic

269. What is Databricks?

Databricks is a cloud-based data and AI platform built around Apache Spark and technologies such as Delta Lake. It supports data engineering, analytics, machine learning, and AI workloads.

270. What is the Databricks Lakehouse?

The Lakehouse combines characteristics of data lakes and data warehouses, providing scalable storage with features such as reliability, governance, and efficient analytical processing.

271. What is Delta Lake?

Delta Lake is a storage layer that adds features such as ACID transactions, schema enforcement, time travel, and reliable updates to data stored in a data lake.

272. Why is Delta Lake useful compared with plain Parquet files?

Plain Parquet provides efficient file-based storage but lacks transaction management and many data-management capabilities. Delta Lake adds transactional reliability, schema management, history, and update/delete support.

273. What is a Databricks cluster?

A Databricks cluster is a collection of compute resources used to execute notebooks, jobs, Spark applications, and other workloads.

274. What is DBFS?

DBFS, or Databricks File System, provides a filesystem-like interface for interacting with cloud storage from Databricks. Modern Databricks architectures often access cloud object storage directly using governed storage mechanisms.

275. What are the main interfaces available in Databricks?

Common interfaces include notebooks, SQL editor, workflows/jobs, dashboards, APIs, and development tools.

276. How would you ingest data into Databricks?

Data can be ingested from cloud storage, databases, streaming systems, APIs, and other sources using Spark, Auto Loader, connectors, or SQL-based mechanisms.

Intermediate

277. Explain the relationship between Databricks, Spark, Delta Lake, and the Lakehouse.

Databricks provides the platform, Spark provides distributed processing, Delta Lake provides reliable storage capabilities, and the Lakehouse architecture combines these capabilities for unified analytics and data engineering.

278. What are the benefits of ACID transactions in Delta Lake?

They provide reliable atomic, consistent, isolated, and durable changes, preventing partial or inconsistent updates when multiple operations interact with data.

279. How does Databricks support scalable data engineering?

It provides distributed Spark processing, autoscaling compute, Delta Lake, workflow

orchestration, cloud storage integration, monitoring, governance, and collaborative development.

280. What is the purpose of Databricks Assistant?

Databricks Assistant is an AI-powered development assistant that helps users write, explain, debug, and work with code and data within Databricks.

281. How would you troubleshoot a slow Databricks job?

Inspect Spark UI and query execution details, identify expensive stages, check shuffles and skew, review partitioning and joins, examine cluster sizing, and optimize transformations and storage access.

282. What factors affect Databricks cluster cost and performance?

Cluster size, instance type, runtime, number of workers, autoscaling configuration, workload duration, data volume, caching, and inefficient processing patterns all affect cost and performance.

Vector Search, RAG and Agentic Applications

Basic

283. What is a vector embedding?

An embedding is a numerical vector representing the semantic characteristics of data such as text, allowing similar concepts to be compared mathematically.

284. What is Vector Search?

Vector Search retrieves data based on similarity between embeddings rather than relying only on exact keyword matches.

285. What is Retrieval-Augmented Generation?

RAG combines information retrieval with a generative AI model. Relevant information is retrieved from a knowledge source and provided to the model to generate a grounded response.

286. Why is RAG useful for enterprise AI applications?

It allows models to use organization-specific and current information without requiring the model itself to contain all that knowledge. It can also improve grounding and reduce unsupported responses.

Intermediate

287. Explain the basic flow of a RAG application.

Documents are collected and chunked, embeddings are generated and stored, the user's query is embedded, relevant chunks are retrieved, and those chunks are provided to the language model to generate the response.

288. How does vector search retrieve relevant information?

The query is converted into an embedding and compared with stored embeddings using a similarity measure. The most similar vectors are returned as relevant results.

289. What is the difference between semantic search and keyword search?

Keyword search focuses on matching words or terms. Semantic search uses embeddings to identify meaning and conceptual similarity even when exact words differ.

290. What is the role of the Mosaic AI Agent Framework?

It provides capabilities for building, evaluating, deploying, and managing AI agents that can use models, retrieval systems, and tools.

291. What are tool functions in an AI agent?

Tool functions are controlled operations that an agent can invoke to retrieve information or perform actions, such as querying a database or calling an API.

292. How would an agent use Unity Catalog tool functions?

An agent can invoke governed tools exposed through Unity Catalog to access approved data or perform approved operations while respecting enterprise permissions and governance.

Advanced**293. How would you build and deploy a retrieval agent on Databricks?**

Ingest and index enterprise data, create embeddings and a retrieval layer, define agent instructions and tools, evaluate retrieval and generation quality, apply governance and permissions, and deploy the agent through the appropriate Databricks serving infrastructure.

294. How would you evaluate whether a RAG system retrieves the right information?

Create representative queries with expected relevant documents and evaluate retrieval metrics such as recall@K and precision@K. Also evaluate whether retrieved context actually supports the generated answer.

295. What problems can cause poor RAG responses, and how would you troubleshoot them?

Poor chunking, weak embeddings, bad metadata, irrelevant retrieval, insufficient context, outdated data, or poor prompts can cause failures. I would separately evaluate ingestion, chunking, retrieval, context construction, and generation.

296. How would you secure an enterprise agent that accesses internal data?

Use identity-based access control, least privilege, governed data access, secure tool interfaces, audit logging, encryption, input/output validation, and protections against prompt injection and unauthorized actions.

297. How would you design an agent that combines document retrieval with external tools?

Give the agent separate tools for retrieval and external actions, define clear permissions, let it decide when each tool is needed, validate tool inputs and outputs, and require confirmation for sensitive actions.

16. DEVOPS AND CONTAINERIZATION

DevOps and DataOps

Basic

298. What is DevOps?

DevOps is a set of practices that combines development and operations to automate software delivery, improve collaboration, and increase deployment reliability.

299. What is DataOps?

DataOps applies DevOps principles to data workflows, focusing on automation, collaboration, testing, data quality, and reliable delivery of data.

300. Why are DevOps practices important for data engineering?

They enable version control, automated testing, CI/CD, repeatable deployments, monitoring, and faster recovery from failures.

301. How is DataOps different from traditional DevOps?

DevOps primarily focuses on software development and operations. DataOps applies similar practices to data pipelines while additionally emphasizing data quality, schema changes, lineage, and data reliability.

302. What is CI/CD?

Continuous Integration automatically builds and tests changes as they are integrated. Continuous Delivery or Deployment automates the process of releasing validated changes to environments.

Intermediate

303. How can CI/CD be applied to data pipelines?

Pipeline code, SQL, configurations, and infrastructure can be version controlled. Automated tests can run on every change before deploying validated pipelines to staging and production.

304. What types of automated tests should be used for data pipelines?

Use unit tests, integration tests, schema tests, data-quality tests, transformation tests, reconciliation checks, and end-to-end tests.

305. What is Infrastructure as Code?

Infrastructure as Code manages infrastructure through version-controlled configuration files rather than manual setup. Examples include Terraform and CloudFormation.

306. Why is versioning important for data engineering projects?

It provides traceability, collaboration, reproducibility, auditing, and the ability to identify or roll back changes.

307. How can DataOps improve data quality and reliability?

Through automated validation, monitoring, testing, version control, lineage, standardized workflows, and rapid detection and recovery from pipeline failures.

Advanced

308. How would you design a CI/CD workflow for a production data pipeline?

A typical workflow would validate code, run unit and data-quality tests, build artifacts, deploy to a test environment, run integration tests, and promote approved changes to production with monitoring and rollback capability.

309. How would you prevent bad code or bad data from reaching production?

Use pull-request reviews, automated tests, schema validation, data-quality gates, staging environments, approval workflows, and deployment checks.

310. How would you handle rollback of a failed data pipeline deployment?

Revert the code or deployment artifact, restore the previous configuration, and use transactional or versioned data mechanisms where possible. For corrupted data, restore or recompute affected datasets from a trusted source.

17. VERSION CONTROL WITH GITHUB

Basic

311. What is Git?

Git is a distributed version control system used to track changes to code and collaborate across development teams.

312. What is GitHub?

GitHub is a platform that hosts Git repositories and provides collaboration features such as pull requests, code reviews, Actions, issues, and project management.

313. What is a Git repository?

A repository is a storage location containing a project's files and its complete Git history.

314. Explain commit, branch, merge, pull, and push.

A commit records changes. A branch provides an independent line of development. Merge combines branches. Pull retrieves changes from a remote repository. Push sends local commits to a remote repository.

315. Why is version control important for data engineering?

It tracks changes to pipeline code, SQL, configurations, and infrastructure, enabling collaboration, auditing, reproducibility, and rollback.

Intermediate

316. What is a pull request, and why is it useful?

A pull request proposes changes for review before they are merged. It enables code review, automated checks, discussion, and controlled integration.

317. How would you resolve a merge conflict?

Identify conflicting files, inspect both versions, decide the correct combined result, edit the files, test the changes, and commit the resolution.

318. What is branching strategy?

A branching strategy defines how teams organize branches for development, features, releases, and production. It ensures predictable collaboration and release management.

319. How would you structure a Git repository for a data engineering project?

Separate pipeline code, SQL, configuration, tests, infrastructure, documentation, and deployment workflows. Keep secrets outside the repository.

320. What should and should not be committed to Git?

Commit source code, SQL, tests, configuration templates, and documentation. Do not commit passwords, API keys, private credentials, large generated files, or sensitive production data.

Advanced

321. How would you implement code review practices for a data engineering team?

Use pull requests with required reviewers, automated CI checks, coding standards, data-quality validation, clear descriptions, and protected production branches.

322. How would you recover from an incorrect production commit?

Identify the problematic change, assess its impact, and revert it using Git or deploy a known-good version. Data changes may require separate recovery or recomputation.

323. How would you manage environment-specific configuration without committing secrets?

Store non-sensitive configuration externally or through environment-specific configuration files, and store secrets in a dedicated secret manager or CI/CD secret store.

18. RELEASE MANAGEMENT

Basic

324. What is Release Management?

Release Management is the process of planning, coordinating, testing, deploying, and monitoring changes released to users or production systems.

325. What is the difference between development, testing, staging, and production environments?

Development is used to build changes. Testing validates functionality. Staging closely resembles production for final validation. Production is the live environment used by real users or systems.

326. What is a release?

A release is a controlled package of software, configuration, database, or pipeline changes delivered to an environment.

327. Why are release processes important?

They reduce deployment risk, improve consistency, provide traceability, and make it easier to detect and recover from failures.

Intermediate

328. What should be included in a production release checklist?

Include tested artifacts, approvals, dependencies, configuration, database changes, backup/recovery plans, monitoring, communication, rollback procedures, and post-release validation.

329. How would you manage versioning between releases?

Use source-control tags or release versions and maintain clear release notes. Versioning should identify exactly which code and configuration were deployed.

330. What is rollback, and when should it be used?

Rollback restores a previous known-good version after a release causes unacceptable issues. It should be used when recovery is safer and faster than fixing forward.

331. How would you reduce deployment risk?

Use automated testing, staging, incremental releases, approvals, monitoring, backups, feature flags where applicable, and well-defined rollback procedures.

Advanced

332. How would you design a reliable release process for data pipelines?

Use version-controlled artifacts, automated tests, staging validation, controlled promotion, dependency checks, data-quality gates, monitoring, and recovery procedures.

333. How would you handle a release that partially succeeds?

Stop further deployment, identify which components succeeded, assess dependencies and data impact, then either roll back affected components or complete the deployment safely.

334. How would you coordinate application, database, and data pipeline changes in one release?

Map dependencies, sequence compatible changes carefully, use backward-compatible schema changes where possible, test the complete flow in staging, and coordinate rollback plans.

19. GITHUB ACTIONS

Basic

335. What is GitHub Actions?

GitHub Actions is a CI/CD automation platform integrated with GitHub repositories.

336. What is a workflow?

A workflow is an automated process defined in a YAML file that runs in response to events or schedules.

337. What are jobs and steps in GitHub Actions?

A job is a group of tasks executed on a runner. Steps are individual commands or actions executed within a job.

338. What triggers can start a GitHub Actions workflow?

Triggers can include pushes, pull requests, schedules, manual execution, releases, and other GitHub events.

Intermediate

339. How would you use GitHub Actions to run automated tests?

Create a workflow triggered by pushes or pull requests, install dependencies, run the test suite, and fail the workflow if tests do not pass.

340. How would you build and deploy a Python or PySpark project using GitHub Actions?

Install dependencies, run linting and tests, package the application, build an artifact or container, authenticate securely, and deploy it to the target platform.

341. How would you manage secrets in GitHub Actions?

Store sensitive values in GitHub's encrypted secrets or an external secret manager. Never hard-code credentials in workflow files.

342. What is a reusable workflow?

A reusable workflow is a GitHub Actions workflow designed to be called by other workflows, allowing teams to standardize repeated CI/CD processes.

Advanced

343. How would you design a CI/CD pipeline using GitHub Actions for a data engineering project?

Trigger on pull requests, run linting and unit tests, validate SQL and data schemas, build artifacts, deploy to a test environment, run integration tests, and promote approved artifacts to production.

344. How would you prevent deployment to production unless tests pass?

Make production deployment dependent on successful test jobs and use protected environments with required approvals where necessary.

345. How would you troubleshoot a GitHub Actions workflow that works locally but fails in CI?

Compare Python/runtime versions, dependencies, environment variables, file paths, permissions, operating-system differences, network access, and CI logs. Reproduce the CI environment locally when possible.

20. DOCKER AND KUBERNETES

Basic

346. What is Docker?

Docker is a platform for building, packaging, and running applications in containers.

347. What is a Docker image?

A Docker image is an immutable package containing an application's code, runtime, libraries, and configuration required to create containers.

348. What is a Docker container?

A container is a running instance of an image. It provides process isolation while sharing the host operating system kernel.

349. What is a Dockerfile?

A Dockerfile is a text file containing instructions used to build a Docker image.

350. Why are containers useful in data engineering?

They provide reproducible environments, consistent dependencies, portability, isolation, and easier deployment of data applications.

351. What is Kubernetes?

Kubernetes is an open-source platform for orchestrating containerized applications across a cluster of machines.

352. What is a Kubernetes Pod?

A Pod is the smallest deployable Kubernetes unit and contains one or more closely related containers that share networking and storage.

353. What is a Kubernetes Deployment?

A Deployment manages a desired number of Pod replicas and handles rolling updates, replacement of failed Pods, and maintaining the desired state.

Intermediate

354. Compare containers and virtual machines.

Virtual machines virtualize complete operating systems and are relatively heavyweight. Containers share the host kernel and are generally lighter and faster to start.

355. How does Kubernetes manage containerized applications?

It schedules Pods, maintains desired state, handles scaling and service discovery, performs rolling updates, and replaces failed containers or Pods.

356. What are Services, Deployments, and ConfigMaps in Kubernetes?

Deployments manage application Pods and updates. Services provide stable network access to Pods. ConfigMaps store non-sensitive configuration values.

357. How would you containerize a Python data processing application?

Create a Dockerfile with a suitable base image, install dependencies, copy application code, define the execution command, build the image, test it, and run it as a container.

358. How would you make a containerized application configurable across environments?

Use environment variables, ConfigMaps, configuration files, or deployment parameters rather than hard-coding environment-specific values into the image.

359. What happens when a Kubernetes Pod fails?

Kubernetes detects that the desired state is not met and typically creates or restarts a replacement Pod depending on the workload controller and failure type.

Advanced

360. What problems does Kubernetes solve for microservices?

It provides scheduling, service discovery, scaling, rolling deployments, self-healing, resource management, and standardized orchestration for containerized services.

361. How would you scale a containerized data processing service?

Scale horizontally by increasing Pod replicas and use Kubernetes autoscaling based on appropriate resource or application metrics. The application itself must support parallel processing.

362. How would you troubleshoot a Kubernetes application that keeps restarting?

Check Pod status and events, container logs, resource limits, health probes, configuration, dependencies, and exit codes. `CrashLoopBackOff` often indicates repeated application startup failures.

363. How would you securely manage secrets in a Kubernetes environment?

Use Kubernetes Secrets or an external secret-management system, restrict access through RBAC, encrypt secrets at rest where supported, and avoid embedding secrets in images or source code.

364. How would AWS and Kubernetes work together to run microservices at scale?

AWS provides infrastructure and managed services, while Kubernetes orchestrates containers. Amazon EKS can provide managed Kubernetes control-plane capabilities while applications run on AWS compute resources.

365. What factors should be considered when deciding whether Kubernetes is appropriate for a data engineering workload?

Consider workload type, scale, team expertise, operational complexity, portability, networking, security, cost, and whether managed serverless or specialized data-

processing services would be simpler. Kubernetes is powerful, but it is not automatically the best choice for every data workload.

9/8/2026, 12:40:13 AM



Yes. These are the **highest-priority 58**, so here are concise, interview-ready answers with enough depth to demonstrate actual understanding.

SQL

1. Explain window functions with a practical example.

A window function performs a calculation across related rows without combining them into a single row like `GROUP BY`.

For example, to rank employees by salary within each department:

```
SELECT employee, department, salary,
       RANK() OVER (
         PARTITION BY department
         ORDER BY salary DESC
       ) AS salary_rank
FROM employees;
```

Here, `PARTITION BY` creates a separate ranking for each department.

2. Difference between RANK, DENSE_RANK, and ROW_NUMBER.

`ROW_NUMBER()` gives every row a unique number. `RANK()` gives tied rows the same rank but skips numbers afterward. `DENSE_RANK()` also gives ties the same rank but does not skip numbers.

For salaries `100, 100, 90`:

- `ROW_NUMBER` : 1, 2, 3
- `RANK` : 1, 1, 3
- `DENSE_RANK` : 1, 1, 2

3. How do you optimize a slow SQL query?

First, inspect the execution plan to find the bottleneck. Then I would check indexes, unnecessary joins, large table scans, filters, data types, sorting, aggregation, and

partitioning. I would also select only required columns and filter data as early as possible.

4. How do indexes improve query performance?

An index creates an optimized lookup structure so the database can find rows without scanning the entire table. It improves reads, especially for filtering and joins, but consumes storage and can slow inserts, updates, and deletes because the index must also be maintained.

5. How would you remove duplicates while retaining the latest record?

Use `ROW_NUMBER()` partitioned by the business key and ordered by the timestamp descending:

```
ROW_NUMBER() OVER (  
    PARTITION BY customer_id  
    ORDER BY updated_at DESC  
)
```

Keep only rows where the row number is `1`.

Python

6. Compare lists, tuples, sets, and dictionaries.

- **List:** Ordered and mutable, allows duplicates.
- **Tuple:** Ordered and immutable, allows duplicates.
- **Set:** Stores unique values and is useful for fast membership checks.
- **Dictionary:** Stores key-value pairs and provides fast lookup by key.

7. How would you process a file larger than available memory?

I would process it incrementally instead of loading it entirely. For example, I could read it line-by-line, use generators, process it in chunks with Pandas, or use a distributed framework such as Spark for very large datasets.

8. How would you optimize slow Pandas code?

I would avoid Python loops and unnecessary `apply()` calls, use vectorized operations, reduce unnecessary columns, optimize data types, process data in chunks, and avoid copying DataFrames unnecessarily. If the data is too large, I would consider Spark or another distributed processing tool.

9. How would you structure a production ETL script?

I would separate the pipeline into extraction, transformation, validation, and loading functions or modules. I would also include configuration management, logging, exception handling, testing, retries where appropriate, and clear entry points.

10. How does Python handle exceptions and logging?

Exceptions are handled using `try`, `except`, `else`, and `finally`. For production pipelines, I would use Python's `logging` module to record execution details, warnings, and errors with timestamps and useful context.

Data Warehousing

11. OLTP vs OLAP.

OLTP systems handle day-to-day transactions and are optimized for fast inserts, updates, and point lookups.

OLAP systems are designed for analytics and complex queries over large amounts of historical data.

Example: A banking transaction system is OLTP, while a sales analytics warehouse is OLAP.

12. Star schema vs snowflake schema.

A **star schema** has a central fact table connected directly to relatively denormalized dimensions. It is simpler and usually easier and faster for analytics.

A **snowflake schema** further normalizes dimensions into multiple tables. It reduces redundancy but introduces more joins and complexity.

13. ETL vs ELT.

ETL: Extract → Transform → Load. Data is transformed before entering the target warehouse.

ELT: Extract → Load → Transform. Raw data is loaded first and transformations happen inside the warehouse or lakehouse.

ELT is common in modern cloud data platforms because they provide significant compute power for transformations.

14. Fact table vs dimension table.

A **fact table** stores measurable business events, such as sales, orders, or transactions.

A **dimension table** stores descriptive information about those events, such as customer, product, location, and date.

15. Explain slowly changing dimensions.

Slowly Changing Dimensions are techniques for handling changes in dimension data over time.

For example, if a customer changes their city:

- **Type 1:** Overwrite the old city.
- **Type 2:** Create a new record and preserve the historical city, usually with effective dates.
- **Type 3:** Store limited previous-value information in additional columns.

16. What is a surrogate key?

A surrogate key is an artificial identifier, usually an integer, generated for a warehouse record.

It is useful because it provides a stable warehouse-specific identifier and makes historical tracking, especially SCD Type 2, easier.

17. How would you model a sales data warehouse?

I would create a central `FactSales` table containing measures such as quantity, revenue, discount, and profit.

It would connect to dimensions such as:

- `DimCustomer`
- `DimProduct`
- `DimDate`
- `DimStore`
- `DimEmployee`

This would typically form a star schema.

PySpark

18. Explain Spark architecture.

A Spark application mainly consists of a **Driver**, **Executors**, and a **Cluster Manager**.

The Driver coordinates the application and creates the execution plan. Executors run tasks and process/store data. The Cluster Manager allocates resources to the application.

19. RDD vs DataFrame.

RDDs are low-level distributed collections that provide more control but require more manual optimization.

DataFrames are structured distributed datasets with named columns. Spark can optimize DataFrame operations using its query optimizer, so they are generally preferred for structured data.

20. Transformation vs action.

A **transformation** creates a new dataset from an existing one, such as `filter`, `select`, or `groupBy`.

An **action** triggers execution and produces a result or writes data, such as `count()`, `collect()`, or `write()`.

21. Lazy evaluation.

Spark does not immediately execute transformations. It builds an execution plan and waits until an action is called.

This allows Spark to optimize the complete execution plan before actually processing the data.

22. Narrow vs wide transformations.

A **narrow transformation** can process each output partition using data from a single input partition, such as `filter`.

A **wide transformation** requires data to move between partitions, such as `groupBy` or many joins. Wide transformations usually cause shuffles.

23. What causes a shuffle?

A shuffle occurs when Spark needs to redistribute data across partitions.

Common causes include `groupBy`, `join`, `distinct`, `orderBy`, and `repartition`.

Shuffles are expensive because they involve network, disk, and serialization overhead.

24. How do you optimize a slow Spark job?

I would first inspect the Spark UI to identify slow stages. Then I would look for excessive shuffles, data skew, inefficient joins, poor partitioning, unnecessary columns, large data movement, and expensive UDFs.

I would consider filtering early, using broadcast joins, optimizing partitions, using efficient formats such as Parquet, and caching only when beneficial.

25. What is data skew?

Data skew occurs when data is distributed unevenly across partitions.

For example, if one customer has 90% of all records, the partition containing that customer's data may take much longer than others, creating a slow "straggler" task.

26. When should you use broadcast joins?

Use a broadcast join when one side of the join is small enough to fit comfortably in executor memory.

Spark distributes the small dataset to executors, allowing them to join locally with the large dataset and potentially avoiding a large shuffle.

27. How does Spark recover from failures?

Spark uses lineage to remember how data was created. If a partition is lost, Spark can recompute that partition from the original data and transformations.

It can also retry failed tasks and rely on the cluster manager to replace failed resources.

Databricks

28. What is a Lakehouse?

A Lakehouse combines the scalability and flexibility of a data lake with capabilities traditionally associated with data warehouses, such as ACID transactions, governance, and reliable analytical processing.

29. What is Delta Lake?

Delta Lake is a storage layer built on cloud object storage that adds capabilities such as ACID transactions, schema enforcement, time travel, and reliable updates and deletes.

30. Why Delta Lake over plain Parquet?

Parquet is an efficient file format, but by itself it does not provide full transactional management or reliable table-level operations.

Delta Lake adds ACID transactions, schema enforcement, version history, updates, deletes, and other data-management capabilities.

31. How would you troubleshoot a slow Databricks job?

I would inspect the Spark UI and identify slow stages, shuffles, skew, expensive joins, and large scans.

Then I would check partitioning, cluster configuration, data layout, query plans, file sizes, caching, and inefficient transformations.

32. Explain RAG.

RAG stands for Retrieval-Augmented Generation.

Instead of relying only on the model's internal knowledge, the system retrieves relevant information from an external knowledge base and provides it to the LLM as context before generating the answer.

33. Explain Vector Search.

Vector Search retrieves information based on semantic similarity.

Documents are converted into numerical embeddings, and a user's query is also converted into an embedding. The system finds stored vectors that are most similar to the query vector.

34. How would you build a retrieval agent?

I would:

1. Ingest and clean the source documents.
2. Chunk the documents.
3. Generate embeddings.
4. Store them in a vector index.
5. Build retrieval logic.
6. Give an AI agent access to the retrieval tool.
7. Provide the retrieved context to the LLM.
8. Evaluate retrieval and answer quality.
9. Add authentication, authorization, monitoring, and safety controls.
10. Deploy it to production.

AWS

35. Explain S3 and its storage classes.

S3 is AWS object storage where data is stored as objects inside buckets.

Common storage classes include:

- **Standard:** Frequently accessed data.
- **Intelligent-Tiering:** Unknown or changing access patterns.
- **Standard-IA:** Infrequently accessed data.
- **Glacier classes:** Long-term archival with lower storage costs.

The choice depends on access frequency, retrieval requirements, durability, and cost.

36. EC2 vs Lambda vs ECS.

EC2: Virtual servers with maximum infrastructure control.

Lambda: Serverless, event-driven execution without managing servers, best suited to relatively short-running tasks.

ECS: Managed container orchestration for running Docker containers.

37. Explain VPC and subnets.

A **VPC** is an isolated virtual network in AWS.

A VPC is divided into **subnets**, which are associated with Availability Zones. Public subnets can have direct routes to the internet through an Internet Gateway, while private subnets typically use NAT for outbound internet access.

38. Explain IAM roles and policies.

IAM controls access to AWS resources.

A **role** is an identity that can be assumed by users, applications, or AWS services. A **policy** defines what actions are allowed or denied on which resources.

Roles are generally preferred for applications because they avoid embedding long-term credentials.

39. How would you secure an S3 data lake?

I would use:

- IAM least-privilege access
- Bucket policies
- S3 Block Public Access
- Encryption at rest
- Controlled encryption-key permissions
- Logging and monitoring
- Versioning where appropriate
- Network controls such as VPC endpoints where applicable
- Data classification and access controls

The main principle is to ensure users and services only get the access they actually need.

40. How would you design a scalable AWS data pipeline?

A typical architecture could be:

Sources → ingestion → S3 data lake → processing → curated S3/warehouse → analytics

I would use partitioned storage, columnar formats such as Parquet, scalable processing, automated retries, monitoring, IAM controls, and autoscaling where appropriate.

I would also design for idempotency, fault tolerance, data-quality validation, and cost optimization.

DevOps

41. DevOps vs DataOps.

DevOps applies automation, CI/CD, testing, monitoring, and collaboration to software development and operations.

DataOps applies similar principles to data workflows, with additional emphasis on data quality, schemas, lineage, pipeline reliability, and data validation.

42. Explain CI/CD.

Continuous Integration automatically builds and tests changes when developers integrate code.

Continuous Delivery/Deployment automates the process of releasing validated changes to environments.

The goal is to make releases faster, repeatable, and safer.

43. What is Infrastructure as Code?

Infrastructure as Code, or IaC, means defining infrastructure using version-controlled configuration files rather than manually creating resources.

Tools such as Terraform and CloudFormation can create and manage infrastructure consistently and reproducibly.

44. Explain Git branching and pull requests.

A branch is an independent line of development used to work on changes without directly modifying the main codebase.

A pull request proposes those changes for review. Automated tests and code reviews can happen before the branch is merged.

45. How would you build CI/CD for a PySpark pipeline?

I would create a GitHub Actions workflow that:

1. Runs on pull requests.
2. Installs dependencies.
3. Runs linting and unit tests.
4. Validates SQL and configurations.
5. Builds the deployment artifact.
6. Deploys to a test environment.

7. Runs integration and data-quality tests.
8. Requires approval if appropriate.
9. Deploys the validated artifact to production.

46. Docker image vs container.

A **Docker image** is the packaged blueprint containing application code and dependencies.

A **container** is a running instance of that image.

Think of an image as the template and a container as the running application created from that template.

47. What problem does Kubernetes solve?

Kubernetes manages containerized applications across multiple machines.

It handles scheduling, scaling, service discovery, rolling deployments, resource management, and recovery of failed containers.

48. How would you troubleshoot a failing Kubernetes Pod?

I would check:

1. Pod status and events.
2. Container logs.
3. Exit codes.
4. CPU and memory limits.
5. Liveness/readiness probes.
6. Environment variables and ConfigMaps.
7. Secrets.
8. Network connectivity.
9. Dependencies.

For repeated crashes, `CrashLoopBackOff` and the container's logs are usually good starting points.

AI / GenAI

49. AI vs ML vs Deep Learning vs Generative AI.

AI is the broad field of building systems capable of intelligent behavior.

ML is a subset of AI where systems learn patterns from data.

Deep Learning is a subset of ML using multi-layer neural networks.

Generative AI refers to models that generate new content such as text, images, audio, video, or code.

50. Supervised vs unsupervised vs reinforcement learning.

Supervised learning: Learns from labeled examples, such as predicting whether a transaction is fraudulent.

Unsupervised learning: Finds patterns in unlabeled data, such as customer clustering.

Reinforcement learning: Learns actions through interaction with an environment and rewards.

51. Explain overfitting.

Overfitting occurs when a model learns the training data too closely, including noise, and therefore performs poorly on unseen data.

It can be reduced using techniques such as regularization, more training data, simpler models, dropout, or early stopping.

52. Explain neural networks and backpropagation.

A neural network consists of layers of interconnected neurons. Each neuron uses weights and biases to transform inputs and activation functions to introduce non-linearity.

During training, **backpropagation** calculates how each parameter contributed to the prediction error, and an optimizer such as gradient descent updates the weights to reduce that error.

53. RNN vs LSTM vs Transformer.

RNN: Processes sequences sequentially and can struggle with long-term dependencies.

LSTM: An improved RNN using gates to retain important information for longer periods.

Transformer: Uses attention rather than recurrence and can process sequence positions more efficiently in parallel during training. Transformers are now dominant for many NLP and GenAI applications.

54. What is self-attention?

Self-attention allows each token in a sequence to determine which other tokens are relevant to it.

For example, when processing a word in a sentence, the model can assign greater attention to other words that help determine its meaning.

55. What is RAG?

RAG, or Retrieval-Augmented Generation, combines retrieval with generation.

The system retrieves relevant information from a knowledge source and gives that information to an LLM as context. This helps the model generate answers grounded in external or enterprise-specific information.

56. What is an AI agent?

An AI agent is a system that can pursue a goal by reasoning about tasks, deciding what to do, using tools, observing results, and taking additional actions.

Unlike a simple model response, an agent can execute multi-step workflows.

57. Agentic AI vs chatbot.

A traditional chatbot mainly responds to user messages.

An agentic AI system can plan and execute multi-step tasks, use external tools, access databases or APIs, maintain state, and adapt its actions based on results.

For example, a chatbot can explain how to book a flight, while an agent could potentially search flights, compare options, and book one through authorized tools.

58. What are the risks of enterprise AI agents?

The major risks include:

- Unauthorized access to sensitive data
- Excessive tool permissions
- Prompt injection
- Incorrect actions
- Data leakage
- Hallucinations
- Privacy and compliance issues
- Poor auditability
- Unintended side effects

The key controls are **least-privilege access, strong authentication, tool-level authorization, validation, monitoring, audit logs, human approval for sensitive actions, and careful evaluation.**

9/8/2026, 12:42:45 AM